



Rapport TP5 - Simulation Supermarché

Conception et Systèmes d'Exploitation

Auteurs: Oscar ISOREZ & Baptiste TUAUX

M1 ISTIC

9 décembre 2025

Table des matières

1	Introduction	3
1.1	Contexte du projet	3
2	Architecture du système	3
2.1	Vue d'ensemble	3
2.2	Les entités principales	3
2.2.1	Client - L'acteur principal	3
2.2.2	Rayon - La ressource partagée	3
2.2.3	Chariots - Le pool de ressources limitées	4
2.2.4	Tapis - Le buffer borné (Pattern Producteur-Consommateur)	4
2.2.5	EmpCaisse - Le consommateur	4
2.2.6	EmpRayon - Le réapprovisionneur	4
3	Mécanismes de synchronisation	5
3.1	Moniteurs	5
3.1.1	Gestion des chariots - variable de comptage	5
3.1.2	Buffer du tapis de caisse - Producteur/Consommateur	5
3.2	Exclusion mutuelle	6
3.2.1	Accès aux rayons	6
3.3	Conditions d'attente	6
3.3.1	Attente de chariot disponible	6
3.3.2	Attente de fin de passage en caisse	6
3.3.3	Attente de produit disponible	6
4	Problèmes rencontrés et solutions	7
4.1	Deadlock sur le Tapis	7
4.2	ConcurrentModificationException	7
4.3	Arrêt propre des threads	8
4.4	Race condition sur les rayons	8
5	Exécution et résultats	8
5.1	Exemple d'exécution	8
5.2	Analyse des logs	9
5.3	Scénarios observés	9
5.4	Statistiques de l'exécution	9
6	Conclusion	9
6.1	Concepts maîtrisés	10
6.2	Points d'amélioration possibles	10
6.3	Bilan	10

1 Introduction

Ce rapport présente l'implémentation d'une simulation de supermarché utilisant la programmation concurrente en Java. L'objectif est de modéliser les interactions entre différents acteurs (clients, employés) et ressources partagées (chariots, rayons, tapis de caisse).

1.1 Contexte du projet

Dans le cadre du TP5 de Conception et Systèmes d'Exploitation, nous devons implémenter une simulation réaliste d'un supermarché où plusieurs clients effectuent leurs courses simultanément. Nous utilisons les concepts vu en cours de threads, synchronisation et le pattern producteur-consommateur.

2 Architecture du système

2.1 Vue d'ensemble

Le système simule un supermarché avec les éléments suivants :

- Un ensemble de **clients** qui font leurs courses
- Des **rayons** contenant différents produits
- Un **pool de chariots** en nombre limité
- Une **caisse** avec un tapis roulant (buffer)
- Des **employés** (caissier et employé de rayon)

2.2 Les entités principales

2.2.1 Client - L'acteur principal

Le client est le thread principal de la simulation. Son cycle de vie est le suivant :

1. **Entrée dans le supermarché** : Le client arrive et génère sa liste de courses aléatoire
2. **Prise d'un chariot** : Attend si aucun chariot n'est disponible (ressource limitée)
3. **Shopping** : Parcourt les rayons pour récupérer les produits de sa liste
 - Attend si un produit n'est pas disponible (rayon vide)
 - Ajoute les produits à son panier
4. **Passage en caisse** : Dépose ses articles sur le tapis
 - Attend si le tapis est plein
 - Attend que tous ses articles soient scannés
5. **Sortie** : Rend le chariot et quitte le supermarché

Attributs principaux :

- `clientId` : Identifiant unique du client
- `listeCourses` : Map des produits souhaités avec quantités
- `panier` : Liste des produits récupérés
- `chariots`, `rayons`, `tapis` : Références aux ressources partagées

2.2.2 Rayon - La ressource partagée

Chaque rayon contient un seul type de produit avec une capacité maximale. C'est une ressource partagée entre :

- Les **clients** qui prélèvent des produits (consommateurs)
- L'**employé de rayon** qui réapprovisionne (producteur)

Attributs principaux :

- `product` : Type de produit (enum)
- `nbProductMax` : Capacité maximale du rayon
- `currentAmountProducts` : Stock actuel

Méthodes synchronisées :

- `pickProducts(amount, clientId)` : Prélèvement atomique par un client
- `refill(amount)` : Réapprovisionnement atomique par l'employé
- `isFull()` : Vérifie si le rayon est plein

2.2.3 Chariots - Le pool de ressources limitées

Le pool de chariots représente une ressource limitée classique. Les clients doivent attendre si tous les chariots sont utilisés.

2.2.4 Tapis - Le buffer borné (Pattern Producteur-Consommateur)

Le tapis de caisse est implémenté comme un **buffer circulaire borné**. C'est l'exemple classique du pattern producteur-consommateur :

- **Producteurs** : Les clients déposent leurs articles
- **Consommateur** : L'employé de caisse retire et scanne les articles

Attributs principaux :

- `tapis[]` : Tableau circulaire d'articles
- `max_articles` : Capacité maximale du buffer
- `start, end` : Pointeurs de lecture/écriture
- `size` : Nombre d'articles actuels
- `checkoutComplete` : Flag de fin de passage en caisse

Mécanisme de synchronisation :

- `ajouterArticle()` : Bloque si le buffer est plein (`wait()`)
- `retirerArticle()` : Bloque si le buffer est vide (`wait()`)
- Utilisation d'un **sentinel** (-1) pour marquer la fin des articles d'un client

2.2.5 EmpCaisse - Le consommateur

L'employé de caisse est un thread qui tourne en boucle et consomme les articles du tapis.

Cycle de vie :

1. Attente d'un article sur le tapis
2. Scan de l'article (simulation avec `sleep`)
3. Si sentinel (-1) détecté : signale la fin du passage en caisse
4. Répète jusqu'à interruption

2.2.6 EmpRayon - Le réapprovisionneur

L'employé de rayon maintient les rayons approvisionnés en effectuant des allers-retours avec l'entrepôt.

Cycle de vie :

1. Vérifie s'il a des produits à transporter
2. Si non : va chercher des produits à l'entrepôt
3. Parcourt les rayons et réapprovisionne ceux qui ne sont pas pleins
4. Répète jusqu'à interruption

Attributs principaux :

- `carriedProducts` : Map des produits transportés
- `MAX_CARRY_PER_PRODUCT` : Capacité de transport par type de produit

3 Mécanismes de synchronisation

3.1 Moniteurs

3.1.1 Gestion des chariots - variable de comptage

Les chariots fonctionnent comme un **sémaphore de comptage** :

```
public class Chariots {
    private int currentNbChariots; // Compteur du sémaphore

    public synchronized void prendreChariot(int clientId) {
        while (currentNbChariots <= 0) {
            log("Client-" + clientId + " attend un chariot...");
            try {
                wait(); // P() - Attente si compteur = 0
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
        }
        currentNbChariots--; // Décrément atomique
        log("Client-" + clientId + " prend un chariot");
    }

    public synchronized void rendreChariot(int clientId) {
        currentNbChariots++; // Incrément atomique
        log("Client-" + clientId + " rend son chariot");
        notifyAll(); // V() - Réveille les threads en attente
    }
}
```

3.1.2 Buffer du tapis de caisse - Producteur/Consommateur

Le tapis implémente le pattern classique avec deux conditions d'attente :

```
public synchronized void ajouterArticle(int article, int clientId) {
    while (size >= max_articles) { // Buffer plein
        try {
            wait(); // Producteur attend
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return;
        }
    }
    tapis[end] = article;
    end = (end + 1) % max_articles;
    size++;
    notifyAll(); // Réveille le consommateur
}

public synchronized int retirerArticle() throws InterruptedException {
    while (size == 0) { // Buffer vide
        wait(); // Consommateur attend
    }
    int article = tapis[start];
    start = (start + 1) % max_articles;
    size--;
    notifyAll(); // Réveille les producteurs
    return article;
}
```

3.2 Exclusion mutuelle

3.2.1 Accès aux rayons

Les rayons sont accédés par plusieurs threads :

- **Clients** : `pickProducts()` pour prélever
- **EmpRayon** : `refill()` pour réapprovisionner

Les deux méthodes sont `synchronized` pour garantir l'atomicité :

```
public synchronized ProductEnum pickProducts(int amount, int clientId) {
    if (amount <= currentAmountProducts) {
        currentAmountProducts -= amount;
        log("Client-" + clientId + " prend " + amount);
        return product;
    }
    return null; // Pas assez de stock
}

public synchronized int refill(int nbproducts) {
    int spaceAvailable = nbProductMax - currentAmountProducts;
    int toAdd = Math.min(nbproducts, spaceAvailable);
    currentAmountProducts += toAdd;
    log("Réapprovisionne +" + toAdd);
    return nbproducts - toAdd; // Retourne le surplus
}
```

3.3 Conditions d'attente

3.3.1 Attente de chariot disponible

Un client doit attendre si tous les chariots sont utilisés :

```
while (currentNbChariots <= 0) {
    wait(); // Attente passive - libère le CPU
}
```

3.3.2 Attente de fin de passage en caisse

Le client attend que l'employé de caisse ait scanné tous ses articles :

```
// Côté Client
public void waitForCheckoutComplete(int clientId) throws InterruptedException {
    while (!checkoutComplete) {
        wait();
    }
    checkoutComplete = false; // Reset pour le prochain client
}

// Côté EmpCaisse - quand il détecte le sentinel -1
if (article == -1) {
    checkoutComplete = true;
    notifyAll(); // Réveille le client
}
```

3.3.3 Attente de produit disponible

Si un rayon est vide, le client attend le réapprovisionnement :

```
ProductEnum pickedProduct = null;
int attempts = 0;
while (pickedProduct == null) {
    pickedProduct = rayon.pickProducts(quantity, clientId);
}
```

```

    if (pickedProduct == null) {
        attempts++;
        if (attempts % 5 == 0) {
            log("Attend réapprovisionnement de " + produit + "...");
        }
        Thread.sleep(200); // Attente active avec pause
    }
}

```

4 Problèmes rencontrés et solutions

4.1 Deadlock sur le Tapis

Problème : La première version de `deposerArticles()` était entièrement synchronized :

```

// VERSION BUGGUÉE
public synchronized void deposerArticles(List<Integer> articles) {
    for (int article : articles) {
        ajouterArticle(article); // Bloque si tapis plein
    }
}

```

Cela causait un **deadlock** :

1. Le client tient le verrou sur Tapis (via `deposerArticles`)
2. `ajouterArticle()` appelle `wait()` car le tapis est plein
3. `EmpCaisse` essaie d'appeler `retirerArticle()` mais ne peut pas acquérir le verrou
4. **Deadlock** : Le client attend de l'espace, l'employé attend le verrou

Solution : Retirer la synchronisation globale et ne synchroniser que les opérations atomiques :

```

// VERSION CORRIGÉE
public void deposerArticles(List<Integer> articles, int clientId) {
    synchronized (this) {
        checkoutComplete = false; // Opération atomique courte
    }
    for (int article : articles) {
        ajouterArticle(article, clientId); // Chaque ajout gère sa propre sync
        Thread.sleep(20);
    }
    ajouterArticle(-1, clientId); // Sentinel
}

```

4.2 ConcurrentModificationException

Problème : Modification de `listeCourses` pendant l'itération :

```

// VERSION BUGGUÉE
for (Map.Entry<ProductEnum, Integer> entry : listeCourses.entrySet()) {
    acheterProduit(entry.getKey(), entry.getValue()); // Modifie listeCourses!
}

```

Solution : Créer une copie de la liste avant l'itération :

```

// VERSION CORRIGÉE
List<Map.Entry<ProductEnum, Integer>> shoppingList =
    new ArrayList<>(listeCourses.entrySet());

for (Map.Entry<ProductEnum, Integer> entry : shoppingList) {
    // Maintenant on peut modifier listeCourses sans problème
    acheterProduit(entry.getKey(), entry.getValue());
}

```

4.3 Arrêt propre des threads

Problème : Les threads employés ne s'arrêtaient pas correctement car l'`InterruptedException` n'était pas gérée :

```
// VERSION BUGGUÉE
catch (InterruptedException ex) {
    // Rien - le thread continue
}
```

Solution : Restaurer le flag d'interruption et sortir de la boucle :

```
// VERSION CORRIGÉE
catch (InterruptedException ex) {
    Thread.currentThread().interrupt(); // Restaure le flag
    break; // Sort de la boucle
}
```

4.4 Race condition sur les rayons

Problème : Sans synchronisation, deux clients pouvaient prélever le même stock :

```
// VERSION BUGGUÉE (non synchronized)
if (amount <= currentAmountProducts) { // Thread 1 vérifie: OK
    // Thread 2 vérifie aussi: OK (mais stock insuffisant pour les deux!)
    currentAmountProducts -= amount; // Résultat: stock négatif!
}
```

Solution : Utiliser `synchronized` pour rendre l'opération atomique.

5 Exécution et résultats

5.1 Exemple d'exécution

```
=====
SIMULATION SUPERMARCHE DEMARREE
Paramètres : 5 clients, 4 rayons, 3 chariots, tapis de 10 articles max
=====
[EmpRayon] Commence son service
[EmpCaisse] Pret a servir les clients
[Client-1] Liste de courses: {BEURRE=1, SUCRE=3, FARINE=1, LAIT=5}
[Client-2] Liste de courses: {BEURRE=4, SUCRE=4, FARINE=4, LAIT=1}
[Client-1] Entre dans le supermarche
[Client-3] Liste de courses: {SUCRE=1, LAIT=2}
[Client-2] Entre dans le supermarche
[Chariots] Client-1 prend un chariot (2/3 dispo)
[Chariots] Client-5 prend un chariot (1/3 dispo)
[Chariots] Client-4 prend un chariot (0/3 dispo)
[Chariots] Client-3 attend un chariot... (0/3 dispo)
[Chariots] Client-2 attend un chariot... (0/3 dispo)
[Rayon BEURRE] Client-1 prend 1 (reste: 4/5)
[Rayon BEURRE] Client-4 prend 3 (reste: 1/5)
[Rayon BEURRE] Client-5 prend 1 (reste: 0/5)
...
[Client-5] Se dirige vers la caisse avec 4 articles
[Tapis] Client-5 depose 4 articles
[EmpCaisse] Scanne: BEURRE (article #1)
[EmpCaisse] Scanne: SUCRE (article #2)
[EmpCaisse] Scanne: FARINE (article #3)
[EmpCaisse] Scanne: FARINE (article #4)
[Tapis] Client-5 a termine son passage en caisse
[Client-5] Paiement termine!
```



```
[EmpCaisse] Fin client (Client-1) - 4 articles scannes
[Chariots] Client-5 rend son chariot (1/3 dispo)
[Chariots] Client-3 prend un chariot (0/3 dispo)
[Client-5] Quitte le supermarché. Achats: {BEURRE=1, SUCRE=1, FARINE=2}
...
[EmpCaisse] Termine. Total articles scannes: 42
=====
SIMULATION TERMINEE
=====
```

5.2 Analyse des logs

Les logs montrent plusieurs comportements intéressants avec les paramètres choisis :

1. **Concurrence des chariots** : Avec seulement 3 chariots pour 5 clients, les Client-3 et Client-2 doivent attendre qu'un chariot se libère
2. **Réapprovisionnement dynamique** : L'employé de rayon effectue plusieurs allers-retours à l'entrepôt pour maintenir les stocks
3. **Tapis saturé** : Avec une limite de 10 articles, les gros paniers (Client-4 avec 12 articles) doivent attendre que la caisse libère de l'espace
4. **Synchronisation à la caisse** : Les clients passent un par un grâce au mécanisme de sentinelle

5.3 Scénarios observés

Scénario 1 : Attente de chariot (ressource limitée)

```
[Chariots] Client-4 prend un chariot (0/3 dispo)
[Chariots] Client-3 attend un chariot... (0/3 dispo)
[Chariots] Client-2 attend un chariot... (0/3 dispo)
...
[Chariots] Client-5 rend son chariot (1/3 dispo)
[Chariots] Client-3 prend un chariot (0/3 dispo)
```

Scénario 2 : Attente de réapprovisionnement

```
[Client-4] Attend reapprovisionnement de SUCRE...
[Rayon SUCRE] Reapprovisionne +4 (stock: 5/5)
[Rayon SUCRE] Client-4 prend 4 (reste: 1/5)
```

Scénario 3 : Tapis plein (buffer borné)

```
[Tapis] Plein! Client-4 attend...
[EmpCaisse] Scanne: LAIT (article #2)
[Tapis] Plein! Client-4 attend...
[EmpCaisse] Scanne: LAIT (article #3)
```

5.4 Statistiques de l'exécution

- **Total articles scannés** : 42 articles
- **Clients servis** : 5 clients avec succès
- **Paniers** :
 - Client-1 : 10 articles (BEURRE=1, SUCRE=3, FARINE=1, LAIT=5)
 - Client-2 : 13 articles (BEURRE=4, SUCRE=4, FARINE=4, LAIT=1)
 - Client-3 : 3 articles (SUCRE=1, LAIT=2)
 - Client-4 : 12 articles (BEURRE=3, SUCRE=4, FARINE=2, LAIT=3)
 - Client-5 : 4 articles (BEURRE=1, SUCRE=1, FARINE=2)

6 Conclusion

Cette implémentation démontre les concepts clés de la programmation concurrente :

6.1 Concepts maîtrisés

- **Threads et leur cycle de vie** : Création, exécution, interruption propre
- **Synchronisation avec moniteurs** : `synchronized`, `wait()`, `notify()`, `notifyAll()`
- **Pattern producteur-consommateur** : Implémentation avec buffer borné
- **Gestion des ressources partagées** : Sémaphores, exclusion mutuelle
- **Problèmes classiques** : Identification et résolution de deadlocks, race conditions

6.2 Points d'amélioration possibles

- Utiliser `java.util.concurrent` (Semaphore, BlockingQueue) pour simplifier le code
- Ajouter plusieurs caisses avec load balancing
- Implémenter une file d'attente à la caisse
- Ajouter des statistiques de performance (temps d'attente moyen, etc.)

6.3 Bilan

Ce TP nous a permis de comprendre en profondeur les mécanismes de synchronisation en Java et les problèmes classiques de la programmation concurrente. La simulation du supermarché est un excellent cas d'étude car elle combine plusieurs patterns (producteur-consommateur, pool de ressources, exclusion mutuelle) dans un contexte réaliste et compréhensible.